

第22回 まとめて命令する:ポリモフィズム

今回のゴール

- `Hero` や `Wizard` のインスタンスを、親クラス型の `Character` の変数・配列に入れられる。
- 拡張 `for` 文で、パーティー全員に同じ命令(`attack`)を送れる。
- 同じ `attack` の呼び出しでも、中身のクラスによって違う処理が動くこと(ポリモフィズム)を説明できる。

前回とのつながり

第21回で、`Character` を受けついで `Hero` と `Enemy` を作った。今回は魔法使い(`Wizard`)を仲間に加える。勇者と魔法使いは別のクラスだが、どちらも `Character` なので、1 つの配列にまとめて「全員、攻撃!」と命令できる。

実習時のフォルダ構成

```
ワークスペース
├─ java22_sample
│   └─ src
│       ├── Character.java
│       ├── Hero.java
│       ├── Wizard.java
│       ├── Enemy.java
│       └─ Main.java
└─ java22_exercise
    └─ src
        ├── (sample と同じ 5 ファイル)
        └─ Warrior.java ← 演習 2 - 3 で自分で作る
```

- 配布ファイル: `プログラム/第22回/sample/`、`プログラム/第22回/exercise/`
- 解答例: `プログラム/第22回/answer/`
- `Character.java`、`Hero.java` は第21回と同じ。`Enemy.java` は第21回の解答例(4 ターンごとの強攻撃あり)。

講義内容

1. 親クラス型の変数に、子クラスのインスタンスを入れる

```
Character c1 = new Hero("勇者", 30, 8, 2);
Character c2 = new Wizard("魔法使い", 20, 5);
```

- `Hero` も `Wizard` も `Character` を受けついでいるので、「`Character` の一種」として `Character` 型の変数に入れられる。
- 変数の型(`Character`)と、実際に入っているインスタンスのクラス(`Hero`、`Wizard`)は別のものとして考える。

2. 同じ命令で、違う動き(ポリモフィズム)

プログラム/第22回/sample/Wizard.java

```
// 魔法使い：HP は低いが、魔法で大きなダメージを与える
public class Wizard extends Character {
    public Wizard(String name, int hp, int power) {
        super(name, hp, power);
    }

    // 魔法使いの攻撃：攻撃力の 2 倍の火の魔法
    @Override
    public void attack(Character target, int turn) {
        int damage = getPower() * 2;
        System.out.println(getName() + "は火の魔法をとなえた！ " + target.getName() + "に" + damage +
            "のダメージ");
        target.damage(damage);
    }
}
```

```
Character[] party = { hero, wizard };

for (Character member : party) {
    member.attack(enemy, turn);
}
```

- `member` の型はどちらも `Character` だが、`member.attack(...)` で実際に動くのは、中身のクラスでオーバーライドされた `attack`。1 人目は `Hero` の攻撃、2 人目は `Wizard` の魔法になる。
- 呼び出す側は、相手が勇者か魔法使いかを気にせず「攻撃しろ」と命令するだけでよい。この性質を **ポリモフィズム(多態性)** という。
- 新しい職業を増やしても、この `for` 文は変えなくてよい。

3. 拡張 for 文

```
for (Character member : party) {
    member.showStatus();
}
```

- 「`party` の要素を 1 つずつ `member` に入れて、くり返す」という意味。添字(`i`)を使わずに全員を処理できる。
- 添字が必要なとき(何番目かを表示する、`%` で順番に選ぶ)は、第17回の `for (int i = 0; ...)` を使う。

配列は、要素の数だけ先に作ってから、後で入れることもできる。

```
Character[] party = new Character[3];    // 3 人分の入れ物を作る（中身はまだ null）
party[0] = hero;
party[1] = wizard;
```

4. できること・できないこと(型で決まる)

```
Character c = new Hero("勇者", 30, 8, 2);
c.attack(enemy, 1);    // OK: Character に attack があるので呼べる（動くのは Hero の attack）
c.useHerb();           // コンパイルエラー: Character 型には useHerb がない
```

- 呼び出せるメソッドは変数の型で決まり、実際に動く中身はインスタンスのクラスで決まる。
- そのため、サンプルでは薬草を使うとき、`Hero` 型の変数 `hero` を使っている。
- `Character` 型の変数に入った勇者に `useHerb` させたいときは、次のように「中身が `Hero` か」を調べて、`Hero` 型に変換(キャスト)する方法もある。今は読んで意味が分かればよい。

```
if (c instanceof Hero) {    // c の中身が Hero なら true
    Hero h = (Hero) c;      // Hero 型として扱う
    h.useHerb();
}
```

5. サンプルを読む

プログラム/第22回/sample/Main.java (変わったところ)

```
Hero hero = new Hero("勇者", 30, 8, 2);
Wizard wizard = new Wizard("魔法使い", 20, 5);
Enemy enemy = new Enemy("オーク", 70, 9, 30);

// 味方パーティー: Hero も Wizard も Character 型の配列に入れられる
Character[] party = { hero, wizard };
```

```
switch (command) {
    case 1:
        // 全員が攻撃する（誰の attack が呼ばれるかは、中身のクラスで決まる）
        for (Character member : party) {
            if (member.isAlive() && enemy.isAlive()) {
                member.attack(enemy, turn);
            }
        }
        break;
```

```
enemy.attack(findTarget(party, turn), turn);
```

```
// パーティーに生きているメンバーが 1 人でもいれば true
static boolean isPartyAlive(Character[] party) {
    for (Character member : party) {
        if (member.isAlive()) {
            return true;
        }
    }
    return false;
}

// 敵がねらうメンバーを決める (ターンごとに順番に。たおれているメンバーは飛ばす)
static Character findTarget(Character[] party, int turn) {
    for (int i = 0; i < party.length; i++) {
        Character member = party[(turn + i) % party.length];
        if (member.isAlive()) {
            return member;
        }
    }
    return party[0];
}
```

- `while` の条件は `isPartyAlive(party) && enemy.isAlive()` になった。
- `findTarget` は、ターン数を人数で割った余りで、ねらうメンバーを順番に変えている。メソッドの戻り値が `Character` 型になっている。

6. 実行する前に予想する

1 を続けて入力する。

- 1 ターン目、オークは誰をねらうか(`turn` は 1、`party.length` は 2)。
- 何ターン目にたおせるか。

7. 実行結果

オークがあらわれた！

--- ターン1 ---

勇者 HP:30/30

魔法使い HP:20/20

オーク HP:70/70

1: 攻撃

2: 薬草を使う

3: にげる

4: ためる

コマンドを選ぶ > 1

勇者の攻撃！ オークに8のダメージ

魔法使いは火の魔法をとこなえた！ オークに10のダメージ

オークの攻撃！ 魔法使いに9のダメージ

--- ターン2 ---

勇者 HP:30/30

魔法使い HP:11/20

```
オーク HP:52/70
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ オークに8のダメージ
魔法使いは火の魔法をとнаえた！ オークに10のダメージ
オークの攻撃！ 勇者に9のダメージ

--- ターン3 ---
勇者 HP:21/30
魔法使い HP:11/20
オーク HP:34/70
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ オークに12のダメージ
魔法使いは火の魔法をとнаえた！ オークに10のダメージ
オークの攻撃！ 魔法使いに9のダメージ

--- ターン4 ---
勇者 HP:21/30
魔法使い HP:2/20
オーク HP:12/70
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ オークに8のダメージ
魔法使いは火の魔法をとнаえた！ オークに10のダメージ

オークをたおした！ 経験値30を手に入れた
```

- 同じ `member.attack(enemy, turn)` の 1 行から、「勇者の攻撃」と「火の魔法」の 2 種類が表示されている。

8. エラーを体験する

1. `Main.java` の `hero.useHerb();` を、`party[0].useHerb();` に変えて保存する。
2. `party[0]` の中身は勇者なのに、エラーになる。`party[0]` の型が `Character` だからである。エラーメッセージに `Character` と書かれていることを確認する。
3. 元に戻す。

演習1 サンプルを実行する

`java22_sample` でサンプルを実行し、予想と結果を比べる。

演習2 パーティーを完成させる

`java22_exercise` を作り、`プログラム/第22回/exercise/` の 5 ファイルを入れる。

演習2-1 魔法使いの攻撃

- まず実行して、魔法使いが「魔法使いの攻撃！ オークに5のダメージ」(`Character` のふつうの攻撃)になっていることを確認する。
- `Wizard` で `attack` をオーバーライドし、攻撃力の 2 倍の「火の魔法」にする。

演習2-2 全滅の判定

- 今の `isPartyAlive` は、勇者が生きているかしか調べていない。拡張 `for` 文で全員を調べ、1 人でも生きていれば `true` を返すようにする。

演習2-3 戦士を仲間にする

- 新しいクラス `Warrior` (戦士)を作る。`Character` を受けつぎ、コンストラクタは名前・HP・攻撃力を受け取る。
- `attack` をオーバーライドし、戦士のおのの一撃！ オークに10のダメージ と表示してダメージを与える。
- `Main` で戦士(HP 40、攻撃力 10)を作り、配列 `party` に加える。

ヒント：

- 配列に入れる順番が、攻撃の順番になる。
- `findTarget` と `isPartyAlive` は、`party.length` や拡張 `for` を使っているなので、3 人になっても変えなくてよい。

完成条件(1 ターン目):

```
--- ターン1 ---
勇者 HP:30/30
魔法使い HP:20/20
戦士 HP:40/40
オーク HP:70/70
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃! オークに8のダメージ
魔法使いは火の魔法をとこなえた! オークに10のダメージ
戦士のおのの一撃! オークに10のダメージ
オークの攻撃! 魔法使いに9のダメージ
```

発展(任意)

- ・パーティー全員の HP の合計を計算して表示する `static int totalHp(Character[] party)` を作る。
- ・HP が一番少ないメンバーを返す `static Character findWeakest(Character[] party)` を作り、敵がその相手をねらうようにする。

確認問題

問1 出力は何か(クラスはサンプルと同じ)。

```
Character c = new Wizard("魔法使い", 20, 5);
c.attack(new Character("的", 50, 1), 1);
```

問2 コンパイルできるか。

```
Character c = new Hero("勇者", 30, 8, 2);
c.useHerb();
```

問3 出力は何か。

```
Character[] party = new Character[3];
System.out.println(party.length + " " + party[0]);
```

■ 答え

- ・問1: 魔法使いは火の魔法をとこなえた! 的に10のダメージ (変数の型は `Character` でも、動くのは中身の `Wizard` の `attack`)
- ・問2: できない (`useHerb` は `Hero` にしかない。呼べるメソッドは変数の型 `Character` で決まる)
- ・問3: `3 null` (入れ物は 3 つ作られたが、まだ何も入っていない)

振り返り

- `Character` 型の配列に、勇者と魔法使いを入れられる理由を説明できるか。
- `member.attack(...)` で、誰の `attack` が動くかを予想できたか。
- 「呼べるメソッドは変数の型、動く中身はインスタンスのクラス」を説明できるか。

次回へのつながり

今の `Character` は、`new Character("謎の人", 10, 1)` のように、職業のない「ただのキャラクター」も作れてしまう。また、新しい職業を作ったときに `attack` を書き忘れても、エラーにならない。次回は、「必ず `attack` を書かせる」しくみを作る。